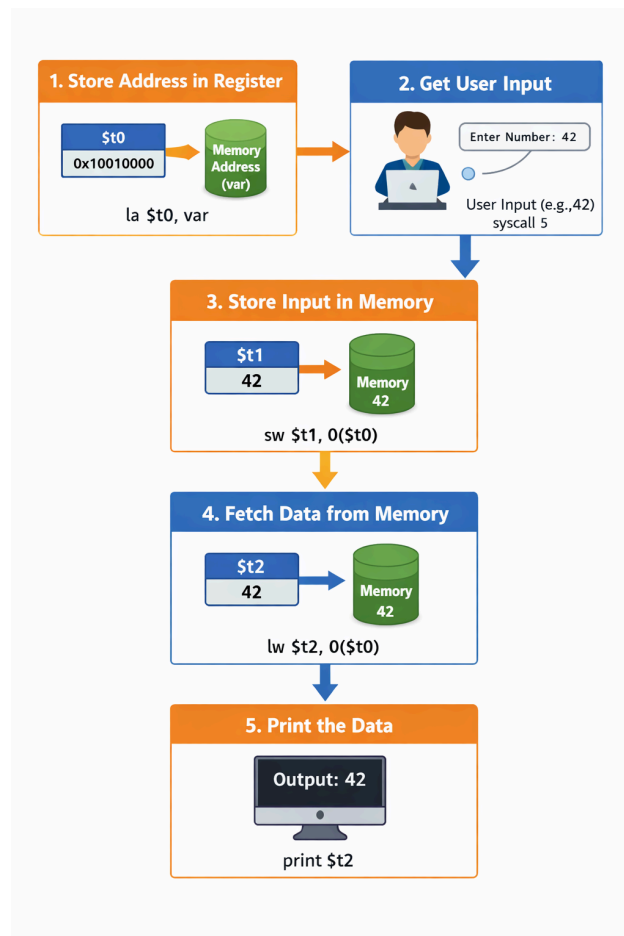
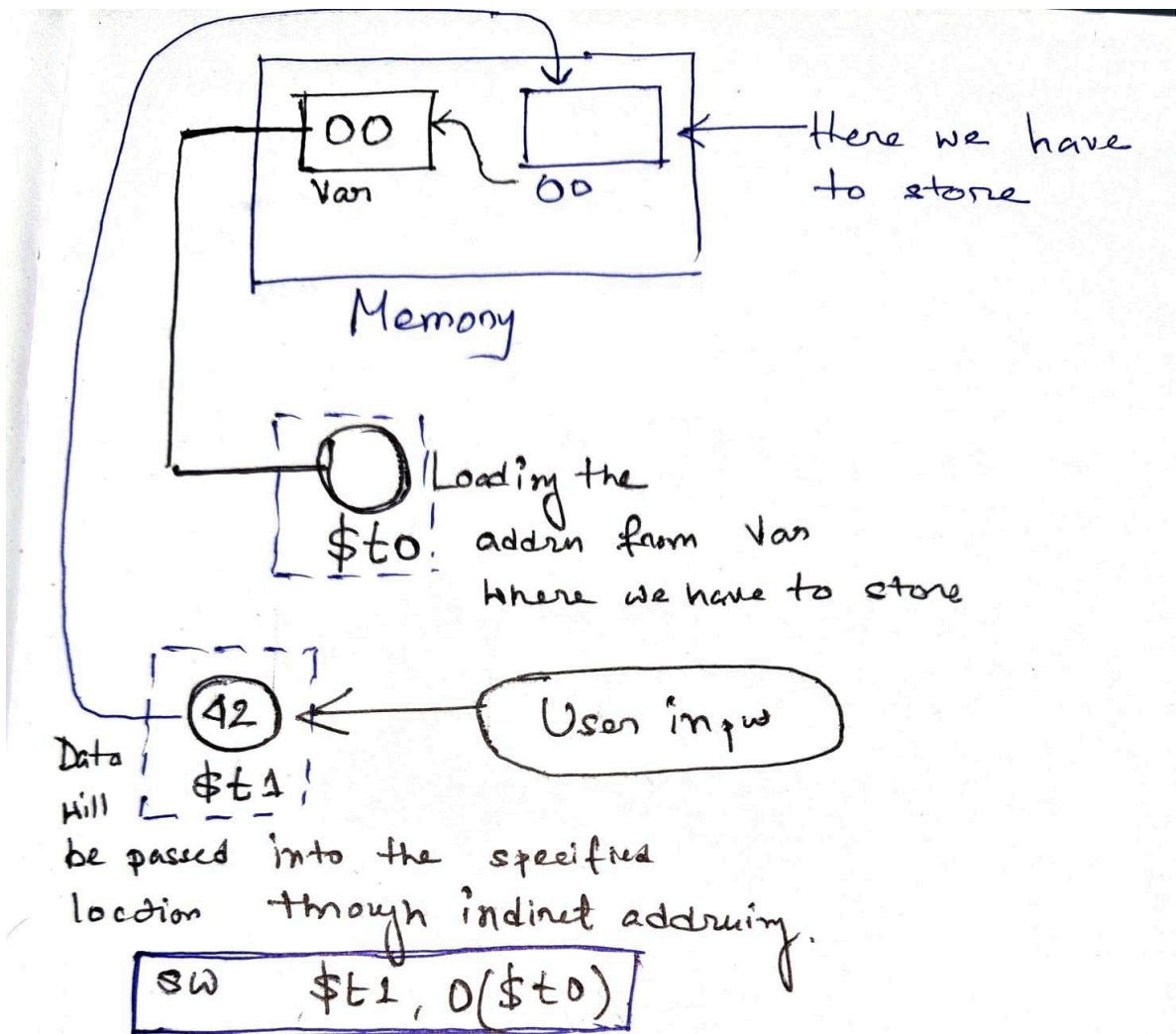


COA Lab 3

Task flow (what we will do)

1. Store a memory address into a register
2. Take user input
3. Store input at that memory address
4. Fetch data from memory
5. Print the fetched data





```
.data
var: .word 0    # memory location to store user input
```

```
.text
.globl main
```

```
main:
    # 1 Store memory address into register
    la $t0, var    # $t0 = address of var
```

```
    # 2 Take user input
```

```

li $v0, 5      # syscall 5 → read integer
syscall
move $t1, $v0  # $t1 = user input

# 3 Store input into memory address
sw $t1, 0($t0) # Memory[var] = user input

# 4 Fetch data from memory
lw $t2, 0($t0) # $t2 = Memory[var]

# 5 Print fetched data
move $a0, $t2
li $v0, 1      # syscall 1 → print integer
syscall

# Exit program
li $v0, 10
syscall

```

Logic (while loop)

```

i = 1
while (i <= 10):
    print i
    i = i + 1

```

MIPS Assembly Code

```

.data
newline: .asciiz "\n"

```

```

.text
.globl main

main:
    li $t0, 1          # i = 1

while:
    bgt $t0, 10, exit  # while (i <= 10)

    # print integer
    li $v0, 1
    move $a0, $t0
    syscall

    # print newline
    li $v0, 4
    la $a0, newline
    syscall

    addi $t0, $t0, 1    # i = i + 1
    j while

exit:
    li $v0, 10          # exit program
    syscall

```

Store **10**, **20**, **30** into an array.

MIPS Code

```

.data
arr: .word 0, 0, 0      # array with 3 integer elements

```

```

.text
.globl main

main:
    la $t0, arr        # base address of array

    li $t1, 10
    sw $t1, 0($t0)     # arr[0] = 10

    li $t1, 20
    sw $t1, 4($t0)     # arr[1] = 20

    li $t1, 30
    sw $t1, 8($t0)     # arr[2] = 30

    li $v0, 10         # exit
    syscall

```

Use of space

```

.data
arr: .space 12    # space for 3 integers (3 × 4 bytes)

```

```

.text
.globl main

```

```

main:
    la $t0, arr    # base address of array

    li $t1, 10
    sw $t1, 0($t0) # arr[0] = 10

    li $t1, 20
    sw $t1, 4($t0) # arr[1] = 20

    li $t1, 30

```

```
sw $t1, 8($t0) # arr[2] = 30
```

```
li $v0, 10    # exit  
syscall
```

◆ Problem Logic (User-defined array)

```
read n  
i = 0  
while i < n:  
    read value  
    array[i] = value  
    i++
```

✓ MIPS Code: User-defined array size & input

```
.data  
arr:      .space 400          # space for max 100 integers (100 × 4 bytes)  
msg1:     .asciiz "Enter array size: "  
msg2:     .asciiz "Enter element: "  
newline:  .asciiz "\n"
```

```
.text  
.globl main
```

```
main:  
    # Ask for array size  
    li $v0, 4  
    la $a0, msg1  
    syscall  
  
    li $v0, 5                # read integer  
    syscall  
    move $t0, $v0            # t0 = n (array size)  
  
    la $t1, arr              # base address of array  
    li $t2, 0                # i = 0
```

```

input_loop:
    bge $t2, $t0, exit    # while (i < n)

    # Prompt for element
    li $v0, 4
    la $a0, msg2
    syscall

    li $v0, 5             # read element
    syscall

    sw $v0, 0($t1)        # arr[i] = input

    addi $t1, $t1, 4      # move to next index
    addi $t2, $t2, 1      # i++

    j input_loop

exit:
    li $v0, 10            # exit program
    Syscall

```

◆ Key instructions used in MIPS functions

Instruction	Purpose
<code>jal</code>	Jump and Link (call function)
<code>jr \$ra</code>	Return from function
<code>\$ra</code>	Stores return address

`$a0–$a3` Function arguments

`$v0–$v1` Return values

`$sp` Stack pointer



MIPS program: Add two numbers using a function

```
.text
.globl main

main:
    # Pass arguments
    li $a0, 10      # first number
    li $a1, 20      # second number
```



```

# Call function
jal add_numbers

# Result is in $v0
move $a0, $v0
li $v0, 1          # print integer
syscall

# Exit
li $v0, 10
syscall

# ----- Function Definition -----
add_numbers:
    add $v0, $a0, $a1    # v0 = a0 + a1
    jr $ra              # return to caller

```



MIPS program: Function with loop (Sum 1 to N)

```

.text
.globl main

main:
    li $a0, 5          # N = 5
    jal sum_to_n       # call function

```

```

# print result
move $a0, $v0
li $v0, 1
syscall

# exit
li $v0, 10
syscall

# ----- Function Definition -----
sum_to_n:
    li $t0, 0          # sum = 0
    li $t1, 1          # i = 1

loop:
    bgt $t1, $a0, end_loop
    add $t0, $t0, $t1 # sum = sum + i
    addi $t1, $t1, 1 # i++
    j loop

end_loop:
    move $v0, $t0      # return sum
    jr $ra

```

Assignment Questions

1. Write a program to print numbers from **1 to 10** using a **while loop**.
2. Write a program that reads an integer **n** from the user and prints all numbers from **1 to n** using a **while loop**.
3. Write a program to store **5 integer values** into an array.

4. Write a program to read **5 elements from the user** and store them in an array using a **while loop**.
5. Write a program to **print all elements of an array** using a **while loop**.
6. Write a program to find the **sum of elements of an array** using a **while loop**.
7. Write a program to **count the number of elements** in an array that are greater than **10**.
8. Write a program to **find the largest element** in an array using a **while loop**.
9. Write a program to copy elements from **one array to another array** using a **while loop**.
10. Write a program that reads the **size of an array from the user** and stores the array elements using a **while loop**.
11. Write a MIPS function to print **Hello World**.
12. Write a MIPS function to find the **square of a number**.
13. Write a MIPS function to print 1 to N (N is user input).
14. Write a MIPS function to **print 1 exactly N times** using a loop.
15. Write a MIPS function to find the answer of $(a+b)^2$.

1 Print numbers from 1 to 10 (while loop)

```
.text
.globl main
main:
    li $t0, 1           # i = 1
```

```
loop1:
    bgt $t0, 10, end1
    move $a0, $t0
    li $v0, 1
    syscall
    addi $t0, $t0, 1
    j loop1
```

```
end1:
    li $v0, 10
    syscall
```

2 Read **n** and print 1 to **n** (while loop)

```
.text
.globl main
main:
    li $v0, 5
    syscall
    move $t1, $v0      # n

    li $t0, 1          # i = 1
```

```
loop2:
    bgt $t0, $t1, end2
    move $a0, $t0
    li $v0, 1
    syscall
    addi $t0, $t0, 1
    j loop2
```

```
end2:
    li $v0, 10
    syscall
```

3 Store 5 integers into an array

```
.data
arr: .word 1,2,3,4,5
```

4 Read 5 elements and store in array (while loop)

```
.data
arr: .space 20

.text
.globl main
main:
    la $t0, arr
    li $t1, 0

loop4:
    bge $t1, 5, end4
    li $v0, 5
    syscall
    sw $v0, 0($t0)
    addi $t0, $t0, 4
    addi $t1, $t1, 1
    j loop4

end4:
    li $v0, 10
    syscall
```

5 Print array elements (while loop)

```
.data
arr: .word 1,2,3,4,5

.text
.globl main
```

```

main:
    la $t0, arr
    li $t1, 0

loop5:
    bge $t1, 5, end5
    lw $a0, 0($t0)
    li $v0, 1
    syscall
    addi $t0, $t0, 4
    addi $t1, $t1, 1
    j loop5

end5:
    li $v0, 10
    syscall

```

6 Find sum of array elements (while loop)

```

.data
arr: .word 1,2,3,4,5

.text
.globl main
main:
    la $t0, arr
    li $t1, 0
    li $t2, 0          # sum

loop6:
    bge $t1, 5, end6
    lw $t3, 0($t0)
    add $t2, $t2, $t3
    addi $t0, $t0, 4
    addi $t1, $t1, 1
    j loop6

```

```
end6:
    move $a0, $t2
    li $v0, 1
    syscall
    li $v0, 10
    syscall
```

7 Count elements greater than 10

```
.data
arr: .word 5,12,8,20,15

.text
.globl main
main:
    la $t0, arr
    li $t1, 0
    li $t2, 0          # count

loop7:
    bge $t1, 5, end7
    lw $t3, 0($t0)
    ble $t3, 10, skip
    addi $t2, $t2, 1

skip:
    addi $t0, $t0, 4
    addi $t1, $t1, 1
    j loop7

end7:
    move $a0, $t2
    li $v0, 1
    syscall
    li $v0, 10
    syscall
```

8 Find largest element in array

```
.data
arr: .word 4,9,2,15,7

.text
.globl main
main:
    la $t0, arr
    lw $t2, 0($t0)    # max
    li $t1, 1
    addi $t0, $t0, 4

loop8:
    bge $t1, 5, end8
    lw $t3, 0($t0)
    ble $t3, $t2, skip8
    move $t2, $t3

skip8:
    addi $t0, $t0, 4
    addi $t1, $t1, 1
    j loop8

end8:
    move $a0, $t2
    li $v0, 1
    syscall
    li $v0, 10
    syscall
```

9 Copy one array to another

```
.data
arr1: .word 1,2,3,4,5
arr2: .space 20
```



```

.text
.globl main
main:
    la $t0, arr1
    la $t1, arr2
    li $t2, 0

loop9:
    bge $t2, 5, end9
    lw $t3, 0($t0)
    sw $t3, 0($t1)
    addi $t0, $t0, 4
    addi $t1, $t1, 4
    addi $t2, $t2, 1
    j loop9

end9:
    li $v0, 10
    syscall

```

10 Read array size and elements (while loop)

```

.data
arr: .space 100

.text
.globl main
main:
    li $v0, 5
    syscall
    move $t1, $v0    # size

    la $t0, arr
    li $t2, 0

loop10:

```

```
        bge $t2, $t1, end10
        li $v0, 5
        syscall
        sw $v0, 0($t0)
        addi $t0, $t0, 4
        addi $t2, $t2, 1
        j loop10

end10:
        li $v0, 10
        syscall
```

PROGRAM 11

Function to print Hello World

```
.data
msg: .asciiz "Hello World\n"

.text
.globl main
main:
    jal print_hello
    li $v0, 10
    syscall

print_hello:
    la $a0, msg
    li $v0, 4
    syscall
    jr $ra
```

PROGRAM 12

Function to find square of a number

```
.text
.globl main
main:
    li $a0, 5
    jal square
    move $a0, $v0
    li $v0, 1
    syscall
    li $v0, 10
    syscall

square:
    mul $v0, $a0, $a0
    jr $ra
```

PROGRAM 13

Function to print 1 to N (user input)

```
.text
.globl main
main:
    li $v0, 5
    syscall
    move $a0, $v0
    jal print_1_to_n
    li $v0, 10
    syscall

print_1_to_n:
    li $t0, 1
loop:
    bgt $t0, $a0, end
    move $a0, $t0
```

```
    li $v0, 1
    syscall
    addi $t0, $t0, 1
    j loop
end:
    jr $ra
```

PROGRAM 14

Function to print 1 exactly N times

```
.text
.globl main
main:
    li $v0, 5
    syscall
    move $a0, $v0
    jal print_n_times
    li $v0, 10
    syscall

print_n_times:
    li $t0, 0
loop:
    bge $t0, $a0, end
    li $a0, 1
    li $v0, 1
    syscall
    addi $t0, $t0, 1
    j loop
end:
    jr $ra
```

PROGRAM 15

Function to find $(a + b)^2$

```
.text
.globl main
main:
    li $a0, 3
    li $a1, 4
    jal square_sum
    move $a0, $v0
    li $v0, 1
    syscall
    li $v0, 10
    syscall

square_sum:
    add $t0, $a0, $a1
    mul $v0, $t0, $t0
    jr $ra
```